



UNISS

UNIVERSITÀ
DEGLI STUDI
DI SASSARI

DIPARTIMENTO DI INGEGNERIA

Corso di Laurea in Ingegneria Informatica

Progetto 17

Generazione di video in lingua dei segni

Andrea FIORI
Matteo PEDONI
Andrea UNALI

GENNAIO 2026

Indice

1	Introduzione	1
2	Studio e Progettazione	2
2.1	Il Dataset di Riferimento: WLASL	2
2.1.1	Composizione e Struttura	2
2.2	Estrazione delle Feature: MediaPipe Holistic	2
2.2.1	Architettura del Modello	3
2.2.2	Topologia dei Landmark	3
2.3	Speech-to-Text	4
2.4	Traduzione Text-to-Gloss	5
2.5	Fondamenti di Elaborazione Numerica e Sintesi Visiva	5
2.5.1	Elaborazione Matriciale e Segnali (NumPy e SciPy)	5
2.5.2	Sintesi Grafica Raster (OpenCV)	5
3	Sforzi Implementativi	7
3.1	Download dei video	7
3.1.1	Protocollo di Download	7
3.1.2	Euristica di Selezione e Ottimizzazione	7
3.1.3	Processo Iterativo e Copertura	7
3.2	Estrazione delle Feature	8
3.2.1	Framework di Rilevamento Pose: MediaPipe Holistic	8
3.2.2	Pre-elaborazione Video: OpenCV	8
3.2.3	Normalizzazione Spaziale (Root Centering)	8
3.2.4	Serializzazione e Gestione dei Dati	9
3.3	Implementazione	9
3.4	Architettura della Pipeline	9
3.5	Modulo di Trascrizione Audio	9
3.6	Elaborazione del Linguaggio Naturale	10
3.7	Architettura e Flusso di Generazione Video	10
3.7.1	Documentazione Tecnica: Classe <code>PoseLoader</code>	10
3.7.2	Visualizzazione e Rendering: Classe <code>HumanoidRenderer</code>	12
3.7.3	Orchestrazione e Sintesi Video: Classe <code>AnimationDirector</code>	14

4	Analisi delle Prestazioni	18
4.1	Ambiente di Test e Metodologia	18
4.1.1	Configurazioni Hardware	18
4.1.2	Metodologia di Misurazione	18
4.2	Risultati Sperimentali	18
4.3	Discussione dei Risultati	19
4.4	Conclusioni sull'Analisi	20
5	Guida per l'Utilizzo	21
5.1	Requisiti di Sistema	21
A	Snippet di Codice	22
A.1	Main : Trascrizione Audio	22
A.2	Main : Generazione Video	22
A.3	Main : Generazione Pose	22
A.4	Main	23
A.5	Main_Test	24
	Elenco delle figure	27
	Bibliografia	28

Capitolo 1

Introduzione

Il presente elaborato descrive la progettazione e la realizzazione di una pipeline di elaborazione dedicata alla generazione automatica di video in lingua dei segni a partire da input testuali o vocali.

L'obiettivo primario del progetto è sviluppare un sistema in grado di tradurre fedelmente un contenuto informativo (audio o testo) in una rappresentazione visiva animata, colmando il divario comunicativo tra udenti e non udenti. Come richiesto dalle specifiche di progetto, il lavoro si articola nell'implementazione di una pipeline di trasformazione input-segno, nella produzione automatica del video tramite tecniche di computer vision e nell'analisi delle prestazioni del sistema.

L'architettura proposta è modulare e si suddivide in tre macro-fasi operative, svolte in concomitanza di una fase di installazione del dataset:

1. **Modulo di Acquisizione e Trascrizione:** Responsabile della conversione dell'input vocale in testo.
2. **Modulo di Elaborazione NLP:** Dedicato all'analisi semantica e sintattica del testo.
3. **Modulo di Sintesi Visiva:** Incaricato della produzione dell'output video finale.

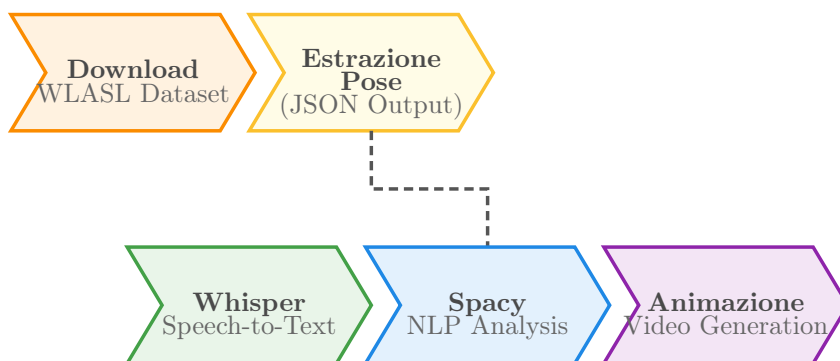


Figura 1.1: Diagramma complessivo del sistema

La pipeline superiore mostra la preparazione del dataset (WLASL e creazione JSON), i cui dati alimentano la pipeline di runtime inferiore (Whisper, Spacy e Animazione). Per la realizzazione di tali componenti, sono state integrate tecnologie allo stato dell'arte quali **OpenAI Whisper**[1] per il riconoscimento vocale, **SpaCy**[2] per l'elaborazione del linguaggio naturale (NLP), e librerie di computer vision come **MediaPipe**[3] e **OpenCV**[4]. Per i dettagli tecnici relativi alla configurazione dell'ambiente di sviluppo e alle dipendenze software, si rimanda al Capitolo 5 (Guida all'installazione).

In merito al dominio linguistico, per ragioni legate alla disponibilità e alla reperibilità di dataset strutturati open-source, si è scelto di adottare l'**American Sign Language (ASL)** come standard di riferimento per la generazione dei segni.

Capitolo 2

Studio e Progettazione

In questo capitolo vengono esposte e motivate la scelta delle tecnologie utilizzate nel progetto.

2.1 Il Dataset di Riferimento: WLASL

Per l'addestramento e la validazione del sistema, la scelta del dataset è ricaduta su **WLASL** (Word-Level American Sign Language), riconosciuto in letteratura come uno dei più vasti e diversificati archivi open-source per il riconoscimento isolato dei segni (ISLR).

2.1.1 Composizione e Struttura

Il dataset WLASL è stato progettato per superare le limitazioni delle raccolte precedenti in termini di varietà lessicale e numero di esecutori. Esso contiene oltre **2.000 classi** (glosses) distinte, per un totale di circa 21.000 video eseguiti da oltre 100 firmatari (signers) differenti [5]. Questa diversità è fondamentale per garantire che il modello generalizzi correttamente e non apprenda caratteristiche specifiche di un singolo individuo.

I metadati del progetto sono organizzati in un file JSON (denominato `WASL_v0.3.json` nella distribuzione utilizzata), che funge da indice strutturato. Per ogni voce lessicale (gloss), il file fornisce:

- **Gloss:** L'etichetta testuale della parola.
- **Instances:** Una lista di video associati a quel segno, contenente URL di origine, ID del firmatario e, in alcuni casi, coordinate di bounding box pre-calcolate.

La natura "in-the-wild" dei video, raccolti da piattaforme come YouTube, introduce sfide significative quali sfondi non uniformi, illuminazione variabile e diverse inquadrature, rendendo necessaria una robusta fase di pre-elaborazione visiva.

2.2 Estrazione delle Feature: MediaPipe Holistic

L'elaborazione diretta dei flussi video grezzi (pixel) richiederebbe una potenza di calcolo proibitiva per un'applicazione real-time. Si è pertanto scelto di astrarre l'informazione visiva convertendola in coordinate spaziali tramite il framework **MediaPipe Holistic**, sviluppato da Google.

2.2.1 Architettura del Modello

MediaPipe Holistic si distingue per il suo approccio a pipeline unificata, ottimizzata per l'inferenza su CPU e dispositivi mobili. Contrariamente ai sistemi che analizzano l'intera immagine per ogni componente anatomico, Holistic adotta una strategia gerarchica:

1. **Stima della Posa (BlazePose):** Il sistema rileva inizialmente la posa corporea generale e i punti chiave del corpo.
2. **Inferenza ROI (Region of Interest):** Sfruttando i punti della posa, l'algoritmo calcola le regioni di interesse (ritagli dell'immagine) dove è probabile che si trovino le mani e il volto.
3. **Dettaglio Locale:** Modelli specifici ad alta risoluzione vengono eseguiti solo su queste regioni ritagliate per estrarre i dettagli fini delle dita e dell'espressione facciale.

Questo approccio riduce drasticamente il carico computazionale, poiché i modelli più pesanti (mani e volto) operano solo su piccole porzioni dell'immagine e solo se la parte del corpo è visibile.

2.2.2 Topologia dei Landmark

Il framework restituisce una topologia completa composta da 57 punti (landmarks) tridimensionali:

- **Pose Landmarks (15 punti):** Scheletro corporeo principale (spalle, gomiti, fianchi).
- **Hand Landmarks (21 punti per ciascuna mano):** Dettaglio articolare essenziale per la chironomia della LIS/ASL.

Ogni landmark è espresso con coordinate (x, y) normalizzate rispetto alle dimensioni dell'immagine, e un valore di *visibility* che indica l'affidabilità della rilevazione. Non si utilizza la dimensione z poiché i video utilizzano una singola angolatura, per cui per questione di spazio l'utilizzo della profondità sarebbe troppo costoso. L'utilizzo pratico di questa tecnologia per la generazione del dataset proprietario sarà dettagliato nel Capitolo 3 (Sforzi Implementativi).

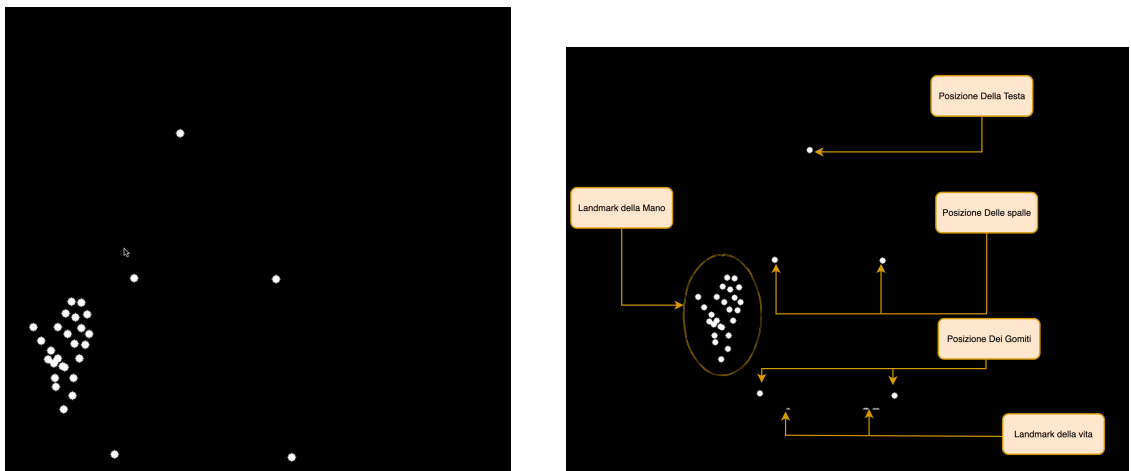


Figura 2.1: Primo plot grezzo dei LandMark estratti.

2.3 Speech-to-Text

Whisper è un sistema di riconoscimento vocale automatico (ASR) addestrato su 680.000 ore di dati supervisionati multilingue e multiuso raccolti dal web. L'utilizzo di un set di dati così ampio e diversificato determina una migliore capacità di gestire accenti, rumori di fondo e linguaggio tecnico. Inoltre, consente la trascrizione in più lingue e la traduzione da queste lingue all'inglese.

L'architettura di Whisper adotta un approccio end-to-end semplice, implementato come un transformer codificatore-decodificatore. L'audio in input viene suddiviso in segmenti di 30 secondi, convertito in uno spettrogramma log-Mel e poi inviato a un codificatore. Un decodificatore viene addestrato a prevedere il testo corrispondente, con token speciali che indicano al singolo modello di eseguire attività come identificazione della lingua, inserimento di indicazioni temporali a livello di frase, trascrizione multilingue del parlato e traduzione del parlato in inglese [1].

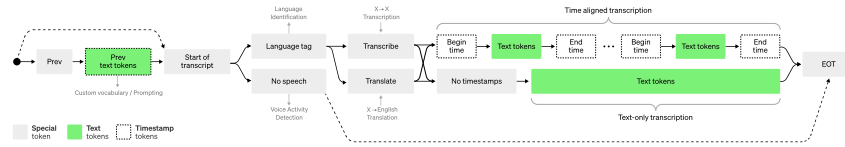


Figura 2.2: Il diagramma mostra le varie fasi della pipeline di whisper[1].

2.4 Traduzione Text-to-Gloss

SpaCy è una libreria avanzata per il Natural Language Processing (NLP) sviluppata in Python e Cython. Basata sui più recenti risultati della ricerca nel settore, l'architettura è stata progettata specificamente sin dal principio per l'utilizzo in contesti produttivi reali e applicazioni industriali.

La libreria fornisce pipeline pre-addestrate (pretrained pipelines) e supporta attualmente la tokenizzazione e l'addestramento per oltre 70 lingue. Caratterizzata da una velocità di esecuzione allo stato dell'arte, spaCy integra modelli basati su reti neurali per l'esecuzione di compiti complessi quali:

- Tagging grammaticale (POS Tagging);
- Analisi sintattica (Parsing);
- Riconoscimento delle entità nominate (NER - Named Entity Recognition);
- Classificazione del testo.

Inoltre, il framework supporta il multi-task learning mediante l'utilizzo di trasformatori pre-addestrati, come BERT. Infine, spaCy offre un sistema di addestramento pronto per la produzione (production-ready), facilitando le operazioni di packaging dei modelli, il deployment e la gestione dei flussi di lavoro [2].

2.5 Fondamenti di Elaborazione Numerica e Sintesi Visiva

La realizzazione di un sistema di generazione video a partire da dati cinematici. Di seguito vengono analizzate le tecnologie e i concetti teorici adottati per la manipolazione dei dati di posa e la loro visualizzazione.

2.5.1 Elaborazione Matriciale e Segnali (NumPy e SciPy)

I dati relativi al movimento umano (coordinate dei giunti nel tempo) possono essere modellati matematicamente come serie temporali multidimensionali o tensori. Per la manipolazione efficiente di tali strutture, lo standard di riferimento in ambiente Python è rappresentato dalla libreria **NumPy**[6], che permette di operare su matrici N -dimensionali ad alte prestazioni.

Tuttavia, i dati provenienti da acquisizioni reali ("in-the-wild") presentano spesso problematiche di discontinuità, come frame rate variabili o dati mancanti (valori *Not-a-Number*). Per risolvere tali inconsistenze, si fa ricorso a tecniche di **Interpolazione Numerica**, implementate nella libreria scientifica **SciPy**[7]. Teoricamente, l'interpolazione mira a stimare nuovi punti dati all'interno dell'intervallo di un insieme discreto di punti noti. Nel contesto del movimento biologico, si distinguono due approcci principali:

- **Interpolazione Lineare:** Collega due punti adiacenti con un segmento rettilineo. È computazionalmente leggera ma produce movimenti rigidi ("robotici") con discontinuità nella velocità (derivata prima).
- **Interpolazione Cubica (Spline):** Utilizza polinomi di terzo grado a tratti per garantire la continuità non solo della posizione, ma anche della velocità e dell'accelerazione. Questo approccio è preferibile per simulare la fluidità naturale delle articolazioni umane.

2.5.2 Sintesi Grafica Raster (OpenCV)

La fase di visualizzazione trasforma le descrizioni matematiche astratte in immagini digitali (pixel). A tale scopo viene utilizzata la libreria **OpenCV** (Open Source Computer Vision)[4], che fornisce primitive grafiche ottimizzate per il disegno raster. Il processo di rendering si basa sul concetto di

Canvas (tela), una matrice numerica $H \times W \times C$ (Altezza, Larghezza, Canali Colore) dove ogni cella rappresenta un pixel. La sintesi video richiede infine un passaggio di **Codifica (Encoding)**, dove la sequenza di matrici/frame viene compressa mediante specifici codec (es. mp4v, H.264) per generare un flusso video fruibile, bilanciando qualità visiva e occupazione di memoria.

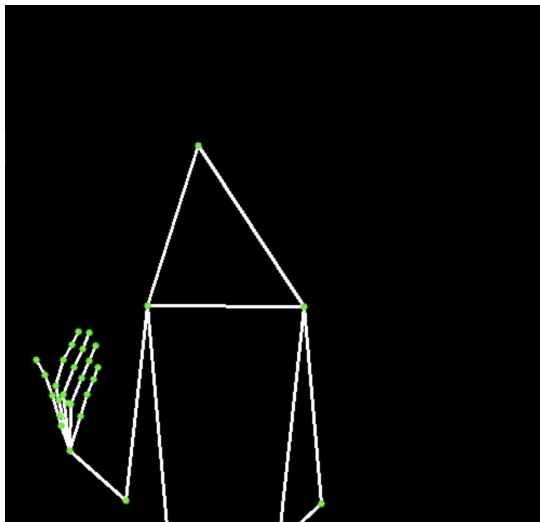


Figura 2.3: Visualizzazione embrionale della struttura: primo test di connessione dei landmark tramite segmenti rettilinei.

Capitolo 3

Sforzi Implementativi

3.1 Download dei video

3.1.1 Protocollo di Download

Il processo di acquisizione ha seguito le linee guida fornite nella repository ufficiale del dataset[8]. La base dati è descritta nel file di metadati `WASL_v0.3.json`, situato nella cartella `start_kit`, che associa ad ogni gloss (etichetta testuale del segno) una lista di URL video corrispondenti a diverse esecuzioni del gesto.

Lo script di download originale fornito nel `start_kit` è stato oggetto di refactoring per rispondere a specifici vincoli di progetto legati all'efficienza dello storage e ai tempi di elaborazione. Data la presenza di molteplici varianti video per il medesimo segno, si è reso necessario implementare una politica di selezione automatica.

3.1.2 Euristica di Selezione e Ottimizzazione

L'obiettivo prioritario è stato minimizzare l'occupazione di memoria senza sacrificare la varietà semantica. A tale scopo, lo script è stato modificato per scaricare, per ogni classe (gloss), esclusivamente l'istanza video con la durata temporale minore. L'ipotesi alla base di questa scelta è che i video più brevi contengono una singola esecuzione della posa, caratteristica fondamentale per semplificare il successivo lavoro di estrazione dei keypoints.

3.1.3 Processo Iterativo e Copertura

L'esecuzione della prima iterazione dello script, basata sul criterio di "durata minima assoluta", ha garantito il recupero dell'96% del dataset totale (i link rimanenti risultavano non più disponibili o corrotti).

Per massimizzare la copertura del dizionario, è stata adottata una strategia di recupero iterativo (fallback):

1. Identificazione dei gloss mancanti dopo la prima passata.
2. Riesecuzione dello script sui gloss mancanti, selezionando il *secondo* video più breve disponibile.
3. Ripetizione del processo fino alla saturazione delle fonti disponibili.

Attraverso questo approccio incrementale, si è raggiunta una copertura finale del **96%** dei gloss previsti. I file scaricati sono stati organizzati nella directory `raw_videos`, che costituisce l'input diretto per la successiva fase di estrazione delle feature tramite MediaPipe.

3.2 Estrazione delle Feature

La fase di acquisizione dati è stata progettata per trasformare i contributi video grezzi in una rappresentazione strutturata e leggera, idonea all'addestramento di modelli di Machine Learning. Di seguito si analizzano i componenti architetturali e le scelte algoritmiche implementate.

3.2.1 Framework di Rilevamento Pose: MediaPipe Holistic

Il nucleo centrale dell'elaborazione (il motore di inferenza) è costituito da **MediaPipe Holistic**. Nello specifico, si è fatto ricorso alla soluzione `mp.solutions.holistic` per le sue capacità di ottimizzazione.

A differenza di approcci tradizionali che richiederebbero l'esecuzione simultanea di tre reti neurali distinte (una per la stima della posa corporea e due per il tracciamento delle mani), Holistic integra questi task in un'unica pipeline unificata. Questo garantisce una maggiore coerenza spaziale tra i segmenti corporei e riduce latenza computazionale.

Il modello restituisce una topologia composta da:

- **Pose Landmarks:** 15 punti chiave rappresentativi del corpo.
- **Hand Landmarks:** 21 punti chiave per ciascuna mano.

Per adattare il sistema alla Lingua dei Segni (ASL/LIS), è stata implementata una logica di filtraggio che isola esclusivamente i *landmark* della parte superiore del corpo (`UPPER_BODY_LANDMARKS`), scartando le informazioni relative a gambe e piedi. Tale scelta riduce il rumore statistico nei dati, focalizzando l'attenzione sulle componenti semantiche rilevanti per la segnatura.

3.2.2 Pre-elaborazione Video: OpenCV

Poiché MediaPipe opera su singoli frame e non su flussi video compressi, la libreria **OpenCV**[4] funge da interfaccia di acquisizione. Le sue funzioni principali all'interno dello script sono:

1. **Decodifica:** Apertura del container video (.mp4) tramite `cv2.VideoCapture` ed estrazione sequenziale dei fotogrammi.
2. **Conversione dello Spazio Colore:** I video vengono nativamente decodificati in formato BGR (Blue-Green-Red). Tuttavia, i modelli pre-addestrati di MediaPipe richiedono input in RGB. La conversione viene gestita puntualmente tramite l'istruzione:

$$Frame_{RGB} = \text{cvtColor}(Frame_{BGR}, \text{COLOR_BGR2RGB}) \quad (3.1)$$

3.2.3 Normalizzazione Spaziale (Root Centering)

Uno degli aspetti più critici nella creazione di dataset per la visione artificiale è l'invarianza posizionale. Senza correzioni, le coordinate dei punti chiave dipenderebbero dalla posizione assoluta del soggetto nell'inquadratura (es. un soggetto a destra avrebbe coordinate x elevate, uno a sinistra coordinate basse), introducendo un *bias* che potrebbe inficiare l'apprendimento del modello.

Per ovviare a questo problema, è stata implementata una tecnica di normalizzazione algoritmica definita **Root Centering**. Il processo si articola nei seguenti passaggi:

1. Calcolo del punto di origine (Root), definito come il punto medio tra i due fianchi (*left_hip* e *right_hip*):

$$C_{root} = \frac{P_{left_hip} + P_{right_hip}}{2} \quad (3.2)$$

2. Traslazione di tutti i punti rilevati (P_i) rispetto alla nuova origine, sottraendo le coordinate del centro:

$$P_{norm}^{(i)} = P_{raw}^{(i)} - C_{root} \quad (3.3)$$

Grazie a questa trasformazione, la testa e le mani sono espresse sempre in relazione al centro del bacino del soggetto (coordinate relative all'origine (0,0)), rendendo il modello robusto rispetto alla posizione dell'utente nello spazio.

3.2.4 Serializzazione e Gestione dei Dati

L'output della pipeline non è costituito da immagini, bensì da file strutturati in formato **JSON**. Questa scelta garantisce interoperabilità e riduce drasticamente lo spazio di archiviazione necessario rispetto ai dati visuali grezzi.

Lo script implementa una logica di *batch processing* sfruttando le librerie di sistema (`os`, `shutil`) per iterare automaticamente su intere directory di video, processarli e archivarli in archivi compressi (.zip). Tale approccio automatizzato è tipico delle moderne pipeline di *Data Engineering*, permettendo la generazione massiva di dataset senza intervento manuale.

3.3 Implementazione

Il nucleo operativo del sistema è governato dallo script `main.py`, il quale implementa una *pipeline* sequenziale dedicata alla trasformazione dell'input audio in una rappresentazione visiva in Lingua dei Segni. L'architettura software proposta integra sinergicamente diverse tecnologie di *Deep Learning* e *Natural Language Processing* (NLP) al fine di gestire le fasi di trascrizione, analisi semantica e generazione dell'animazione.

3.4 Architettura della Pipeline

Il flusso di lavoro, orchestrato dalla funzione `main()`, consente all'utente di selezionare un file audio da una directory sorgente predefinita. L'elaborazione del dato avviene attraverso tre stadi fondamentali:

1. **Speech-to-Text (STT)**: Trascrizione del segnale vocale in testo grezzo.
2. **Text Processing & Mapping**: Analisi linguistica, filtraggio grammaticale e associazione tra lemmi e pose.
3. **Video Rendering**: Generazione della sequenza video finale.

3.5 Modulo di Trascrizione Audio

Per la conversione del segnale vocale in testo, è stato integrato il modello **OpenAI Whisper**. La funzione `audio_to_text`(Codice: A.1) è responsabile del caricamento del modello e dell'esecuzione dell'inferenza.

- **Selezione del Modello**: È stata adottata la versione **base** del modello Whisper. Tale configurazione rappresenta un compromesso ottimale tra accuratezza della trascrizione e velocità di esecuzione, garantendo l'usabilità del sistema anche su architetture hardware con risorse computazionali limitate.
- **Compatibilità Formati**: Il sistema è stato progettato per supportare un ampio spettro di formati audio (tra cui `.mp3`, `.wav`, `.flac`, `.ogg`), assicurando flessibilità nelle fasi di acquisizione dei dati di input.

3.6 Elaborazione del Linguaggio Naturale

A seguito della trascrizione, la funzione `gen_pose` (Codice: A.3) elabora la stringa testuale mediante la libreria **SpaCy** (modello `en_core_web_sm`) [9]. Questa fase è critica per convertire l'inglese parlato in una sequenza di glosse compatibile con la Lingua dei Segni, applicando necessarie regole di semplificazione sintattica.

Il processo di elaborazione segue un algoritmo decisionale iterativo applicato a ciascun *token* rilevato:

- **Filtraggio del Rumore:** Vengono rimossi automaticamente la punteggiatura e gli spazi bianchi, elementi ridondanti ai fini della traduzione in segni.
- **Riduzione Grammaticale (Auxiliary Filtering):** Sfruttando il *Part-of-Speech (POS) Tagging*, il sistema identifica e rimuove i verbi ausiliari (tag *AUX*), quali "be", "do" o "can". Tale scelta implementativa mira a replicare la struttura essenziale tipica delle lingue dei segni, caratterizzate dall'omissione degli ausiliari.
- **Gestione dei Nomi Propri (Fingerspelling):** Qualora un token venga identificato come nome proprio (*PROPN*), l'algoritmo attiva una routine di *fingerspelling*. Il termine viene scomposto nelle singole lettere costituenti, le quali vengono successivamente mappate alle corrispondenti pose dell'alfabeto manuale.
- **Lemmatizzazione:** Per i termini standard, il sistema non utilizza la parola flessa (es. "running"), bensì ne estrae il **lemma** (es. "run") in formato maiuscolo. Ciò garantisce la corretta associazione al file di posa presente nel database, indipendentemente dalla coniugazione verbale o dalla declinazione presente nell'audio originale.

3.7 Architettura e Flusso di Generazione Video

L'ingegnerizzazione del sottosistema di sintesi visiva è stata incapsulata in una libreria proprietaria dedicata, denominata `animation_lib`. L'architettura software adotta il principio della "Separation of Concerns", suddividendo le responsabilità di gestione dati, rendering grafico e orchestrazione in tre classi distinte che cooperano per trasformare i dati grezzi in sequenze fluide.

Operativamente, la fase conclusiva della pipeline è gestita dalla funzione `gen_vid` (rif. Codice A.2). Questa routine agisce da punto di ingresso: riceve in input la lista di glosse (o lettere) prodotta dallo stadio NLP e istanzia la classe `AnimationDirector`. Il sistema verifica la disponibilità dei file di posa pre-calcolati (JSON) nella directory dedicata e assembla i frame corrispondenti in un unico file video di output (formato `.mp4`), mantenendo un riferimento al nome del file audio originale per garantirne la completa tracciabilità. Nelle sottosezioni successive si analizza il funzionamento e le caratteristiche delle classi di `animation_lib`.

3.7.1 Documentazione Tecnica: Classe PoseLoader

Obiettivi e Responsabilità

La classe `PoseLoader` è un componente fondamentale della pipeline di preprocessing, progettato per astrarre la complessità della gestione dei dati grezzi. I suoi obiettivi principali sono:

- **Astrazione I/O:** Gestire il caricamento da file system e il parsing di file JSON con formattazioni eterogenee (tipiche del dataset WLASL).
- **Normalizzazione Spaziale:** Mappare i giunti (joints) presenti nel file sorgente su uno scheletro standardizzato, garantendo che ogni frame abbia lo stesso numero e ordine di feature.
- **Normalizzazione Temporale:** Convertire sequenze registrate a frame rate variabili (spesso bassi, es. 10 fps) in un frame rate target costante (es. 20 fps) necessario per la generazione video fluida.

Struttura dei Dati

Input: La classe accetta in ingresso il percorso di un file `.json`. La struttura interna del file può variare, ma generalmente consiste in una lista di oggetti *frame*, dove ogni oggetto contiene:

- **frame:** indice temporale del fotogramma.
- **joint:** nome identificativo del giunto (es. "nose", "left_wrist").
- **x, y:** coordinate spaziali 2D normalizzate o in pixel.

Output: Il metodo principale restituisce una tupla contenente:

1. **Matrice delle Pose (M):** Un tensore NumPy di dimensione $(T_{target} \times N_{joints} \times 2)$, dove:
 - T_{target} : numero totale di frame interpolati.
 - N_{joints} : numero fisso di giunti monitorati (inclusi corpo e mani).
 - 2: coordinate (x, y) .
2. **Lista Nomi:** Una lista di stringhe che funge da legenda per mappare l'indice della seconda dimensione della matrice al nome anatomico del giunto.

Costruttore

`__init__(self, source_fps=10, target_fps=20)`

Inizializza l'istanza definendo i parametri di campionamento temporale.

- **source_fps:** Il frame rate stimato dei dati di input. Questo valore è cruciale per calcolare la durata temporale corretta della sequenza originale.
- **target_fps:** Il frame rate desiderato per l'output. Determina la densità della matrice risultante.

Descrizione dei Metodi

`load_sequence(file_path)`

Punto di ingresso pubblico della classe. Verifica l'esistenza del file, esegue il caricamento del JSON gestendo eventuali variazioni nello schema (es. liste piatte vs dizionari annidati) e delega l'elaborazione al metodo interno `_process_data`.

`_process_data(frames_data)`

Esegue il parsing strutturale.

1. Organizza i dati grezzi in un dizionario indicizzato per numero di frame.
2. Definisce la topologia dello scheletro target (elenco `target_joints`), che include i giunti corporei base e le dita di entrambe le mani.
3. Costruisce una matrice sparsa iniziale riempiendola con i dati disponibili e lasciando NaN per i dati mancanti.
4. Invoca `_interpolate` per riempire i vuoti e normalizzare il tempo.

`_smooth_signal(data, window_size)`

Implementa un filtro passa-basso semplice (Media Mobile) per ridurre il rumore ad alta frequenza (jitter) sulle coordinate. Gestisce i bordi del segnale replicando i valori estremi per evitare distorsioni all'inizio e alla fine della sequenza.

Approfondimento Tecnico: Algoritmo di Interpolazione Gerarchica

Il metodo `_interpolate` è il nucleo matematico della classe. Non si limita a una semplice interpolazione lineare dei punti, che causerebbe deformazioni anatomiche (es. le dita che si "staccano" o si "comprimono" quando il braccio si muove velocemente). Implementa invece una logica in quattro fasi:

1. **Definizione della Gerarchia:** Viene costruita dinamicamente una mappa delle dipendenze padre-figlio. Specificamente, i giunti delle dita vengono identificati come "figli" dei rispettivi polsi (*wrists*).
2. **Trasformazione in Coordinate Locali:** Prima di interpolare, il sistema converte le coordinate assolute delle dita in coordinate relative rispetto al polso.

$$P_{finger}^{local}(t) = P_{finger}^{global}(t) - P_{wrist}^{global}(t)$$

Questo passaggio è critico: isola il movimento intrinseco della mano dal movimento del braccio nello spazio.

3. **Interpolazione e Smoothing:** Viene calcolato il nuovo asse temporale. Sui dati in spazio locale viene applicata:
 - *Interpolazione Lineare:* Per stimare le posizioni nei nuovi istanti temporali t' . Viene usato un padding dei valori ai bordi (il dito mantiene l'ultima posizione relativa nota se il dato finisce).
 - *Smoothing:* Viene applicata la media mobile descritta in `_smooth_signal`.
4. **Ricostruzione Globale:** Infine, le coordinate vengono riportate nello spazio globale sommando la nuova traiettoria interpolata del polso:

$$P_{finger}^{global}(t') = P_{finger}^{local}(t') + P_{wrist}^{global}(t')$$

Questo approccio garantisce che la struttura rigida della mano venga preservata anche durante i movimenti rapidi, producendo un'animazione finale significativamente più naturale.

3.7.2 Visualizzazione e Rendering: Classe HumanoidRenderer

Obiettivi e Responsabilità

La classe `HumanoidRenderer` è il motore grafico del progetto, responsabile della traduzione dei dati numerici astratti (le coordinate dei giunti) in una rappresentazione visiva antropomorfa. I suoi obiettivi primari sono:

- **Astrazione Grafica:** Isolare la logica di disegno (chiamate a primitive OpenCV) dal resto dell'applicazione.
- **Feedback Visivo:** Fornire un output immediato per il debugging, permettendo di verificare visivamente la qualità dell'interpolazione e la correttezza delle pose.
- **Miglioramento Estetico:** Arricchire lo scheletro base ("stickman") con elementi procedurali (viso, spessore degli arti, muscolatura del trapezio) per rendere il video risultante più naturale e gradevole.

Struttura dei Dati

Input: Il metodo di rendering principale riceve un dizionario di coordinate (*joints_dict*) e un'immagine buffer.

- I dati sono strutturati come mappe { "nome_giunto": (x, y) }, dove x e y sono valori float normalizzati o pre-scalati (dipendente dallo step precedente).
- Il buffer immagine è un array NumPy di dimensioni ($H \times W \times 3$) di tipo `uint8` (canali BGR).

Output: La classe opera in modalità *in-place*, modificando direttamente il buffer dell'immagine fornito per disegnare la posa corrente. Non restituisce nuove strutture dati, ma produce l'effetto collaterale grafico sul frame video.

Costruttore

`__init__(self, width=800, height=600)`

Configura l'ambiente grafico e definisce le proprietà stilistiche statiche:

- Imposta la risoluzione del canvas (W, H).
- Inizializza la **palette colori** tematica (sfondo scuro, pelle chiara, vestiario grigio) e la mappa colori specifica per le dita (codifica cromatica per distinguere visivamente le dita e la mano destra/sinistra durante il debug).
- Definisce i vincoli geometrici, come la lunghezza massima consentita per un osso (`max_bone_len`) e gli spessori di linea variabili per simulare la profondità.

Descrizione dei Metodi

`create_canvas()`

Genera e restituisce una matrice NumPy vuota (tinta unita con colore di background `C_BG`), pronta per essere usata come frame base.

`_gp(joints, name)`

Acronimo di *Get Point*. È il metodo di trasformazione geometrica: recupera le coordinate grezze di un giunto, applica un fattore di scala (1.3x) e un offset per centrare la figura nel canvas. Include un controllo di *clipping* per scartare punti che cadrebbero troppo oltre i margini visibili.

`draw_frame(img, joints_dict, pose_name)`

Il metodo orchestratore ("Director"). Coordina l'ordine di disegno per gestire correttamente la sovrapposizione dei livelli (z-ordering): prima disegna il collo e il busto (livelli posteriori), poi gli arti, e infine la testa e le mani (livelli anteriori). Gestisce anche l'overlay testuale.

`_draw_limb(img, joints, u, v, th)`

Disegna un segmento corporeo ("osso") tra due giunti u e v . Utilizza linee con estremità arrotondate e antialiasing (`cv2.LINE_AA`) per migliorare la resa estetica e simulare il volume cilindrico dell'arto.

`_draw_hand(img, joints, side)`

Itera sulla topologia definita delle dita (dal pollice al mignolo). Per ogni dito, traccia le falangi applicando il colore specifico definito nella palette e verificando la coerenza geometrica (lunghezza ossa).

Approfondimento Tecnico: Ricostruzione Procedurale e Validazione

La classe non si limita a unire i punti ("connect the dots"), ma implementa due logiche avanzate per migliorare il realismo e la robustezza:

1. Generazione Procedurale della Geometria Mancante Il dataset WLASL fornisce solo i punti chiave (es. naso, spalle), ma non la forma del corpo. Il metodo `draw_frame` inferisce geometricamente le aree mancanti:

- **Muscolo Trapezio:** Viene calcolato interpolando il 25% della distanza tra il collo (derivato dal naso) e le spalle, creando un poligono convesso che simula l'attaccatura anatomica del collo.
- **Viso Stilizzato:** Poiché i keypoints facciali sono limitati (spesso solo il naso), gli occhi e la bocca vengono disegnati a offset fissi relativi alla posizione del naso, creando un avatar espressivo senza richiedere dati aggiuntivi.

2. Validazione Euristica della Mano (Sanity Check) Nel metodo `_draw_hand`, è implementato un controllo di integrità sui dati:

$$||P_{curr} - P_{prev}|| < \text{max_bone_len}$$

Prima di disegnare una falange, l'algoritmo calcola la distanza euclidea tra i due giunti. Se questa supera una soglia prefissata (es. 150 px), il disegno del dito viene interrotto. Questo meccanismo funge da filtro visivo per errori di tracciamento (es. un dito che "schizza" via a causa di rumore nel modello di stima), mantenendo il rendering pulito anche in presenza di dati parzialmente corrotti.

3.7.3 Orchestrazione e Sintesi Video: Classe AnimationDirector

Obiettivi e Responsabilità

La classe `AnimationDirector` rappresenta il livello più alto dell'architettura software proposta. Funge da controllore (pattern *Orchestrator*), coordinando l'interazione tra il caricamento dati e il motore di rendering. I suoi obiettivi principali sono:

- **Gestione della Pipeline:** Automatizzare il flusso di lavoro che va dalla lettura dei file JSON alla scrittura del file video MP4.
- **Sintesi di Transizioni:** Risolvere il problema della discontinuità visiva tra due segni diversi generando proceduralmente dei fotogrammi di raccordo (blending), rendendo il passaggio tra una parola e l'altra fluido e naturale.
- **Gestione I/O Video:** Configurare e gestire il codec video e il buffer di scrittura tramite le API di OpenCV.

Struttura dei Dati

Input: Il metodo principale accetta una lista di stringhe (`pose_names`), che rappresentano gli identificativi univoci delle sequenze da concatenare (es. ["ciao", "mondo"]). La classe assume che per ogni nome esista un corrispondente file JSON nella directory configurata.

Output: La classe produce due tipi di output:

1. **File Video:** Un file `.mp4` codificato, contenente l'animazione completa a 30 fps.
2. **Anteprima Real-time:** Una finestra GUI che mostra il frame correntemente elaborato, utile per il monitoraggio visivo durante processi di rendering lunghi.

Costruttore

`__init__(self, poses_dir, output_file="output_sequence.mp4")`

Prepara l'ambiente di lavoro istanziando i componenti dipendenti:

- Inizializza un'istanza di `PoseLoader` configurata a 30 FPS per garantire fluidità.
- Inizializza un'istanza di `HumanoidRenderer` per il disegno.
- Definisce i parametri di transizione, in particolare `transition_frames = 10`, che determina la durata (circa 0.33s) del movimento di raccordo tra due segni consecutivi.

Descrizione dei Metodi

`_create_transition(start_pose, end_pose, steps)`

Metodo interno che implementa la logica di interpolazione. Dati l'ultimo frame della posa precedente e il primo della posa successiva, genera una lista di N frame sintetici intermedi calcolati matematicamente.

`render_sequence(pose_names)`

Il ciclo principale dell'applicazione.

1. Itera sulla lista dei nomi forniti, caricando e validando i dati JSON tramite il `loader`.
2. Inizializza il `cv2.VideoWriter`.
3. Per ogni sequenza caricata:
 - Se esiste una sequenza precedente, invoca `_create_transition` per generare e renderizzare il ponte di collegamento.
 - Renderizza i frame effettivi della sequenza corrente.
 - Memorizza l'ultimo frame corrente per utilizzarlo come punto di partenza per la transizione successiva.
4. Gestisce il rilascio delle risorse e la chiusura dei file.

Approfondimento Tecnico: Interpolazione Lineare (LERP) dei Vettori

Uno degli aspetti più sofisticati di questa classe è la generazione di "frame sintetici" per evitare il *jump-cut* (taglio netto) tra due video diversi, che risulterebbe visivamente fastidioso.

Il metodo `_create_transition` applica un'interpolazione lineare vettoriale su tutti i giunti contemporaneamente. Dato un vettore di posa iniziale $\mathbf{P}_{start}$ e uno finale $\mathbf{P}_{end}$, il frame intermedio al passo i è calcolato come:

$$\mathbf{P}_i = \mathbf{P}_{start} \cdot (1 - \alpha) + \mathbf{P}_{end} \cdot \alpha$$

Dove α è un coefficiente scalare che varia linearmente da 0 a 1 nell'intervallo $[0, 1]$ generato da `numpy.linspace`.

Dal punto di vista implementativo, è stato necessario escludere gli estremi dell'intervallo (0 e 1 puri) per evitare la duplicazione di frame identici all'inizio e alla fine del raccordo, garantendo così una continuità temporale perfetta senza "micro-pause" nell'animazione.

Evoluzione della Resa Visiva

Partendo dalla logica di elaborazione dei dati precedentemente descritta, l'output grafico ha subito un processo di affinamento iterativo. Di seguito vengono analizzate le fasi salienti che hanno portato alla definizione del modello finale.

1. **Prototipo Scheletrico:** Questa iterazione iniziale è stata sviluppata esclusivamente per scopi di debug. L'obiettivo era validare il corretto posizionamento dei *landmark* nello spazio e verificare la coerenza anatomica delle connessioni durante il movimento.

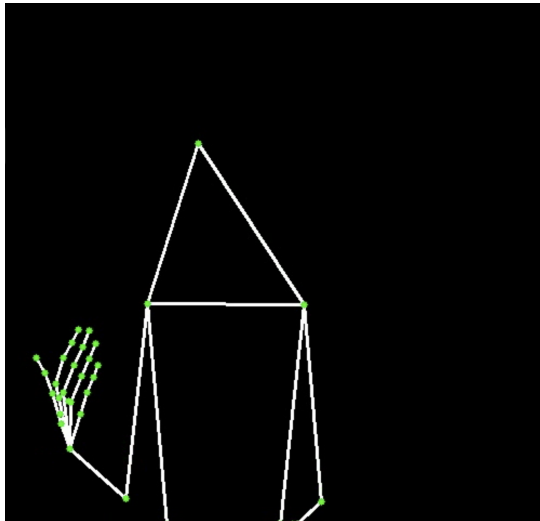


Figura 3.1: Rappresentazione embrionale: test preliminare di interconnessione dei landmark tramite segmenti lineari.

2. **Modello Antropomorfo Intermedio:** In questa fase è stata introdotta una mesh più complessa per favorire la riconoscibilità dei segni. Nonostante il miglioramento visivo, il sistema presentava ancora criticità strutturali:

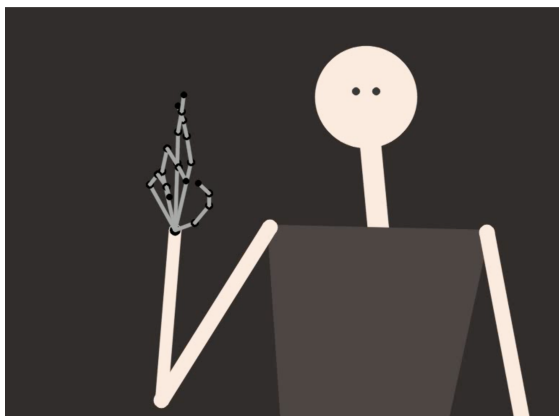


Figura 3.2: Seconda iterazione: transizione verso una forma antropomorfa.

- **Occlusione e sovrapposizione:** Data la mancata gestione della profondità nelle zone di contatto tra mani e dita rendeva l'interpretazione dei segni ambigua.
- **Sproporzioni anatomiche:** Evidenti incongruenze nelle lunghezze relative del collo e delle falangi.
- **Instabilità cinematica:** Alcune configurazioni spaziali causavano *jittering* (scatti improvvisi) o deformazioni innaturali delle estremità.

3. **Risultato Finale:** La versione definitiva, pur mantenendo un'estetica minimalista per garantire alte prestazioni in tempo reale, integra una serie di ottimizzazioni mirate a massimizzare la leggibilità dei gesti.



Figura 3.3: Versione finale dell'avatar ottimizzata per la comunicazione gestuale.

- **Caratterizzazione del volto:** L'aggiunta di tratti somatici distintivi per rendere più piacevole la visione dell'avatar.
- **Codifica cromatica delle dita:** Ad ogni dito è stato assegnato un colore univoco. Questo accorgimento riduce drasticamente l'ambiguità visiva durante le sovrapposizioni e facilita la distinzione immediata tra le dita.
- **Ottimizzazione dei volumi:** L'aumento del diametro delle falangi e dei nodi articolari ha reso i segni più comprensibili.
- **Vincoli cinematici (Constraints):** È stato implementato un limite massimo alla lunghezza dei segmenti, impedendo allungamenti innaturali.
- **Feedback testuale in tempo reale:** L'integrazione del nome della posa riconosciuta a video ha semplificato la fase di test della pipeline e la validazione del sistema di classificazione.

Capitolo 4

Analisi delle Prestazioni

In questo capitolo vengono presentati i risultati sperimentali relativi alle prestazioni della pipeline sviluppata. L'obiettivo dell'analisi è valutare il comportamento del sistema su architetture hardware eterogenee, identificando i colli di bottiglia computazionali e confrontando l'efficienza dei moduli principali: trascrizione audio (Whisper), elaborazione del linguaggio naturale (spaCy) e rendering video.

4.1 Ambiente di Test e Metodologia

Per garantire una valutazione rappresentativa, i test sono stati eseguiti su tre configurazioni hardware distinte, coprendo diverse generazioni di processori e architetture (x86_64 e ARM64).

4.1.1 Configurazioni Hardware

- **Configurazione A (Low-Power x86):** Processore Intel Core i5-8265U (8^a Gen), architettura x86_64.
- **Configurazione B (High-Performance x86):** Processore Intel Core i7-12700H, architettura x86_64 ibrida (P-cores/E-cores).
- **Configurazione C (Apple Silicon):** Processore Apple M1 (2020), architettura ARM64.

4.1.2 Metodologia di Misurazione

Il software di benchmarking è stato integrato direttamente nel codice sorgente (`main_test.py`) (Codice: A.5), utilizzando le librerie `time` e `psutil` per raccogliere metriche in tempo reale. Per ogni fase della pipeline sono stati misurati:

- **Wall Time (T_{real}):** Il tempo totale trascorso percepito dall'utente.
- **CPU Time (T_{cpu}):** Il tempo totale di utilizzo della CPU (che può superare il Wall Time in caso di esecuzione multi-thread).
- **Thread Count:** Il numero di thread attivi associati al processo durante l'esecuzione.

4.2 Risultati Sperimentali

Di seguito vengono riportati i tempi di esecuzione rilevati per ciascun modulo sulle tre configurazioni analizzate.

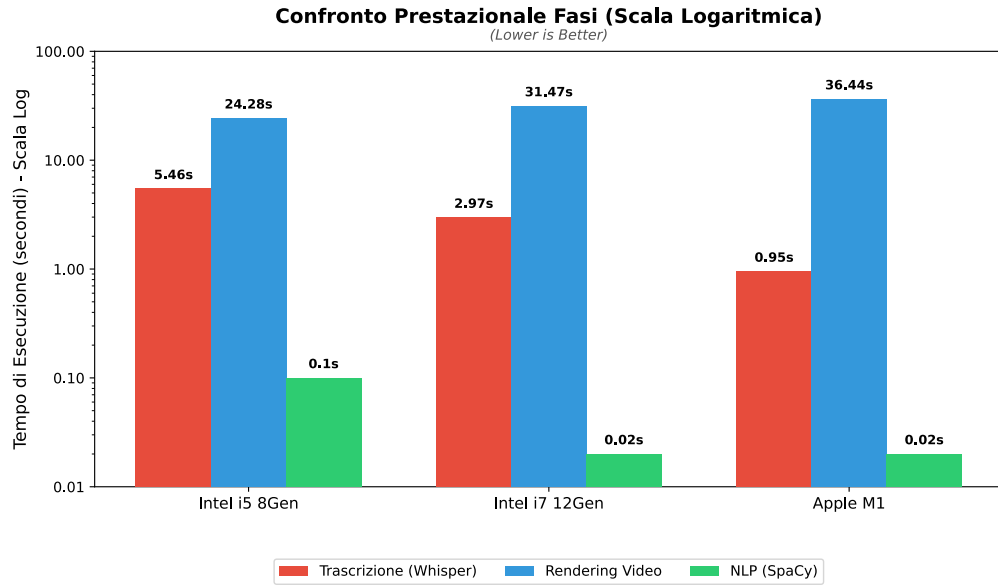


Figura 4.1: Confronto dei tempi di esecuzione su scala logaritmica. Si noti come l’NLP (verde) sia ordini di grandezza più veloce delle altre fasi.

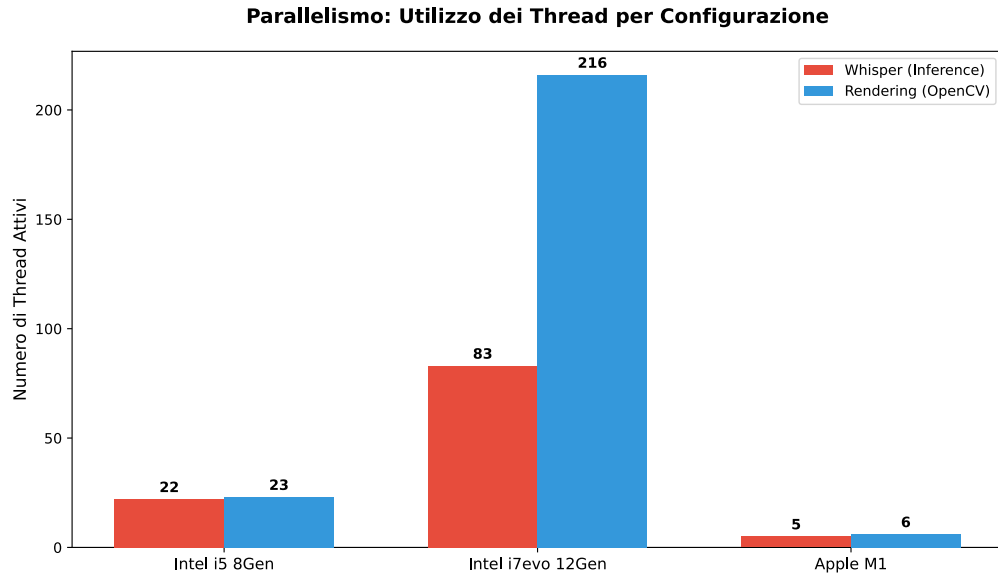


Figura 4.2: Confronto del numero di thread attivi per configurazione hardware.

4.3 Discussione dei Risultati

L’analisi dei dati, riassunti nei grafici 4.1 e 4.2, evidenzia comportamenti divergenti tra i moduli di Intelligenza Artificiale e quelli di pura elaborazione grafica.

Modulo Whisper (ASR) Il processore Apple M1 (Conf. C) ha mostrato prestazioni nettamente superiori nell’inferenza del modello Whisper, completando il task in soli 0.95s, contro i 2.97s dell’i7-12700H e i 5.46s dell’i5-8265U. Questo risultato suggerisce un’ottimizzazione efficace delle istruzioni SIMD su architettura ARM o un utilizzo più efficiente della memoria unificata. Si nota inoltre che l’i7, pur essendo un processore potente, ha registrato un utilizzo massiccio di risorse (CPU Time elevato e picchi di 83 thread) per ottenere un tempo reale superiore a quello dell’M1.

Modulo Rendering Contrariamente alle aspettative, il modulo di rendering video ha ottenuto le prestazioni migliori sulla Configurazione A (i5-8265U), con un tempo di 24.28s. Le configurazioni più moderne sono risultate più lente. Un dato interessante è l'elevato numero di thread osservati sulla Configurazione B (216 thread), che potrebbe indicare un overhead eccessivo nella gestione del multi-threading o una gestione non ottimale delle librerie grafiche sui core ibridi.

Modulo NLP (spaCy) L'elaborazione del testo tramite *spaCy* ha un impatto trascurabile sul tempo totale in tutte le configurazioni, confermando che l'analisi linguistica non rappresenta un collo di bottiglia per il sistema.

4.4 Conclusioni sull'Analisi

In conclusione, la pipeline mostra caratteristiche di prestazione asimmetriche:

1. L'architettura **Apple Silicon (M1)** eccelle nei task di Machine Learning, offrendo la migliore esperienza per la trascrizione vocale.
2. L'architettura **Intel x86** (in particolare nella configurazione i5) mantiene un vantaggio nella fase di rendering video, probabilmente dovuto alla maturità delle librerie grafiche su questa piattaforma.
3. Per un utilizzo in tempo reale, l'ottimizzazione futura dovrà concentrarsi sul modulo di rendering, che rappresenta attualmente la parte preponderante del tempo di esecuzione totale.

Capitolo 5

Guida per l'Utilizzo

La seguente guida illustra come utilizzare il progetto sul vostro calcolatore.

5.1 Requisiti di Sistema

E' necessario aver installato sulla propria macchina Python[10].

Dopo aver installato le dipendenze seguendo le istruzioni contenute nel file **README.md** è possibile provare il progetto. Se si vuole utilizzare la modalità Audio, bisogna prima inserire il file Audio all'interno della cartella, così da poterlo selezionare durante l'esecuzione, mentre per quanto riguarda la modalità testo è possibile inserire l'input durante l'esecuzione .

Appendice A

Snippet di Codice

A.1 Main : Trascrizione Audio

```
1 import whisper
2
3 def audio_to_text(file_path):
4     model = whisper.load_model("base")
5
6     print(f"Inizio elaborazione di: {file_path}")
7     result = model.transcribe(file_path)
8
9     print("Testo rilevato:", result["text"])
10    return result["text"]
```

Codice A.1: Funzione utilizzo Whisper

A.2 Main : Generazione Video

```
1 from animation_lib import AnimationDirector
2
3 def gen_vid(sequence_pose, namefile):
4     # Inizializzazione del motore di rendering
5     director = AnimationDirector(
6         poses_dir=DIR_POSE,      # Directory dei dati di posa
7         output_file=namefile     # File video di output
8     )
9
10    print("Renderizzazione del video...")
11    # Avvio del processo di sintesi frame-by-frame
12    director.render_sequence(sequence_pose)
```

Codice A.2: Astrazione della generazione video tramite AnimationDirector

A.3 Main : Generazione Pose

```
1 def gen_pose(text):
2     """
3     Trasforma il testo in una sequenza di glosse.
4     Filtra gli ausiliari (AUX) e scompone i nomi propri (PROPN) in lettere.
5     """
6     doc = nlp(text)
7     lista_glosse = []
8
9     # Set dei file disponibili per il controllo rapido
10    pose_disp = {
```

```

11     os.path.splitext(f)[0].upper()
12     for f in os.listdir(DIR_POSE)
13 }
14
15 for token in doc:
16     # 1. Salta punteggiatura e spazi
17     if token.is_punct or token.is_space:
18         continue
19
20     # 2. FILTRO AUSILIARI: Salta verbi come 'be', 'do', 'can'
21     if token.pos_ == "AUX":
22         continue
23
24     # 3. NOMI PROPRI: Esegue il fingerspelling lettera per lettera
25     if token.pos_ == "PROPN":
26         for lettera in token.text:
27             char_upper = lettera.upper()
28             if char_upper in pose_disp:
29                 lista_glosse.append(char_upper)
30             else:
31                 print(f"Lettera {char_upper} non trovata per il nome {token.text}")
32     else:
33         # 4. PAROLE STANDARD: Usa il lemma della parola
34         glossa = token.lemma_.upper()
35         if glossa in pose_disp:
36             lista_glosse.append(glossa)
37         else:
38             print(f"Posa non disponibile per: {glossa}")
39
40 if not lista_glosse:
41     raise ValueError("Nessuna glossa valida generata dal testo fornito.")
42
43 return lista_glosse

```

Codice A.3: Generazione delle pose con utilizzo di SpaCy

A.4 Main

```

1 def main():
2     print("\n--- Pipeline Animazione LIS ---")
3     chosetype = input("1. Audio\n2. Testo\n--> scelta: ")
4
5     if not chosetype.isdigit() or int(chosetype) not in (1, 2):
6         print("Scelta non valida.")
7         return
8
9     text_to_process = ""
10    output_filename = ""
11
12    if int(chosetype) == 1:
13        # MODALITA AUDIO
14        audio_files = get_audio_files(AUDIOIO)
15        if not audio_files:
16            print(f"Nessun file audio trovato nella cartella {AUDIOIO}.")
17            return
18
19        for i, f in enumerate(audio_files, start=1):
20            print(f"{i}. {f}")
21
22        choice = input("\nSeleziona file (numero): ")
23        if not choice.isdigit() or not (1 <= int(choice) <= len(audio_files)):
24            print("Scelta non valida.")
25            return
26
27        selected_audio = os.path.join(AUDIOIO, audio_files[int(choice) - 1])
28        output_filename = f"video_{os.path.splitext(os.path.basename(selected_audio))[0]}.mp4"
29
30        # Ottenimento testo tramite Whisper
31        text_to_process = audio_to_text(selected_audio)

```

```

32
33     else:
34         # MODALITA TESTO MANUALE
35         text_to_process = input("\nInserisci la frase da animare (linguaggio naturale): ")
36         if not text_to_process.strip():
37             print("Errore: Testo vuoto.")
38             return
39         output_filename = "video_da_testo.mp4"
40
41     # Fase Comune: Conversione in glosse e Rendering video
42     try:
43         pose_sequence = gen_pose(text_to_process)
44         gen_vid(pose_sequence, output_filename)
45         print("\nProcesso completato con successo.")
46     except Exception as e:
47         print(f"\nErrore durante l'elaborazione: {e}")
48
49 if __name__ == "__main__":
50     main()

```

Codice A.4: Main code

A.5 Main_Test

```

1     import os
2
3
4     import whisper
5     import animation_lib
6     from animation_lib import AnimationDirector
7     import spacy
8     import time
9     import psutil
10
11     # Caricamento del modello linguistico
12     nlp = spacy.load("en_core_web_sm")
13
14     AUDIO = "INPUT_AUDIO"
15     DIR_POSE = "Pose"
16
17     audio_format_valid = ['.mp3', '.wav', '.flac', '.ogg', '.opus', '.m4a', '.aac']
18
19     def get_thread_count():
20         """Ritorna il numero di thread attivi per il processo corrente."""
21         return psutil.Process().num_threads()
22
23     def gen_pose(text):
24         """Trasforma il testo in glosse (spaCy) con benchmark e filtri."""
25         start_wall = time.perf_counter()
26         start_cpu = time.process_time()
27         threads_start = get_thread_count()
28
29         doc = nlp(text)
30         lista_glosse = []
31         pose_disp = {os.path.splitext(f)[0].upper() for f in os.listdir(DIR_POSE)}
32
33         for token in doc:
34             if token.is_punct or token.is_space:
35                 continue
36             if token.pos_ == "AUX":
37                 print(f"[INFO] Salto ausiliare: {token.text}")
38                 continue
39             if token.pos_ == "PROPN":
40                 print(f"[INFO] Nome proprio: {token.text}. Fingerspelling...")
41                 for lettera in token.text:
42                     char_upper = lettera.upper()
43                     if char_upper in pose_disp:
44                         lista_glosse.append(char_upper)
45             else:

```

```

46         glossa = token.lemma_.upper()
47         if glossa in pose_disp:
48             lista_glosse.append(glossa)
49         else:
50             print(f"[WARNING] Posa mancante: {glossa}")
51
52     end_wall = time.perf_counter()
53     end_cpu = time.process_time()
54     print(f"[BENCHMARK] spaCy -> Reale: {end_wall - start_wall:.4f}s | CPU: {end_cpu - start_cpu:.4f}s | Threads: {threads_start}")
55
56     if not lista_glosse:
57         raise ValueError("Nessuna posa valida generata.")
58     return lista_glosse
59
60 def gen_vid(sequence_pose, namefile):
61     """Rendering del video con benchmark."""
62     director = AnimationDirector(poses_dir=DIR_POSE, output_file=namefile)
63     print(f"Renderizzazione del video: {namefile}...")
64
65     start_wall = time.perf_counter()
66     start_cpu = time.process_time()
67     threads_start = get_thread_count()
68
69     director.render_sequence(sequence_pose)
70
71     end_wall = time.perf_counter()
72     end_cpu = time.process_time()
73     print(f"[BENCHMARK] Rendering -> Reale: {end_wall - start_wall:.4f}s | CPU: {end_cpu - start_cpu:.4f}s | Threads: {threads_start}")
74
75 def audio_to_text(file_path):
76     """Trascrizione Whisper con benchmark."""
77     s_load = time.perf_counter()
78     model = whisper.load_model("base", device="cpu")
79     print(f"[BENCHMARK] Load Whisper: {time.perf_counter() - s_load:.4f}s")
80
81     start_wall = time.perf_counter()
82     start_cpu = time.process_time()
83     threads_peak = get_thread_count()
84
85     result = model.transcribe(file_path, fp16=False)
86
87     end_wall = time.perf_counter()
88     end_cpu = time.process_time()
89     print(f"[BENCHMARK] Whisper -> Reale: {end_wall-start_wall:.4f}s | CPU: {end_cpu-start_cpu:.4f}s | Threads: {threads_peak}")
90     print("Testo rilevato:", result["text"])
91     return result["text"]
92
93 def get_audio_files(path):
94     if not os.path.isdir(path): return []
95     return [f for f in os.listdir(path) if os.path.splitext(f)[1].lower() in
96             audio_format_valid]
97
98 def main():
99     print("\n--- Pipeline Animazione LIS ---")
100     chosetype = input("1. Audio\n2. Testo\n--> scelta: ")
101
102     if not chosetype.isdigit() or int(chosetype) not in (1, 2):
103         print("Scelta non valida.")
104         return
105
106     text_to_process = ""
107     output_filename = ""
108
109     if int(chosetype) == 1:
110         # LOGICA AUDIO
111         audio_files = get_audio_files(AUDIO)
112         if not audio_files:
113             print(f"Nessun file in {AUDIO}")
114             return

```

```

114         for i, f in enumerate(audio_files, start=1): print(f"{i}. {f}")
115         choice = input("\nSeleziona file: ")
116         if not choice.isdigit() or not (1 <= int(choice) <= len(audio_files)): return
117
118         selected_audio = os.path.join(AUDIO0, audio_files[int(choice) - 1])
119         output_filename = f"video_{os.path.splitext(os.path.basename(selected_audio))[0]}.mp4"
120
121         # Esecuzione Whisper
122         text_to_process = audio_to_text(selected_audio)
123
124     else:
125         # LOGICA TESTO MANUALE
126         text_to_process = input("\nInserisci la frase da animare: ")
127         if not text_to_process.strip():
128             print("Errore: Testo vuoto.")
129             return
130         output_filename = "video_da_testo.mp4"
131
132     # Fase comune: spaCy -> Rendering
133     try:
134         total_start = time.perf_counter()
135
136         pose_sequence = gen_pose(text_to_process)
137         gen_vid(pose_sequence, output_filename)
138
139         print(f"\n[BENCHMARK TOTALE]: {time.perf_counter() - total_start:.4f}s")
140     except Exception as e:
141         print(f"Errore durante l'elaborazione: {e}")
142
143 if __name__ == "__main__":
144     main()

```

Codice A.5: Main_Test

Elenco delle figure

1.1	Diagramma complessivo del sistema	1
2.1	Primo plot grezzo dei LandMark estratti.	3
2.2	Il diagramma mostra le varie fasi della pipeline di whisper[1].	4
2.3	Visualizzazione embrionale della struttura: primo test di connessione dei landmark tramite segmenti rettilinei.	6
3.1	Rappresentazione embrionale: test preliminare di interconnessione dei landmark tramite segmenti lineari.	16
3.2	Seconda iterazione: transizione verso una forma antropomorfa.	16
3.3	Versione finale dell'avatar ottimizzata per la comunicazione gestuale.	17
4.1	Confronto dei tempi di esecuzione su scala logaritmica. Si noti come l'NLP (verde) sia ordini di grandezza più veloce delle altre fasi.	19
4.2	Confronto del numero di thread attivi per configurazione hardware.	19

Bibliografia

- [1] OpenAI. «Introducing Whisper.» Accessed: 2026, visitato il giorno 2026. indirizzo: <https://openai.com/it-IT/index/whisper/>
- [2] Explosion. «spaCy: Industrial-strength Natural Language Processing in Python.» Accessed: 2026, visitato il giorno 2026. indirizzo: <https://github.com/explosion/spaCy>
- [3] Google. «MediaPipe repo,» visitato il giorno 2026. indirizzo: <https://github.com/google-ai-edge/mediapipe>
- [4] OpenCv. «OpenCv repo,» visitato il giorno 2026. indirizzo: <https://github.com/opencv/opencv>
- [5] D. Li, C. Rodriguez, X. Yu e H. Li, «Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison,» in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2026, pp. 1459–1469.
- [6] T. Oliphant. «NumPy Documentation,» visitato il giorno 2026. indirizzo: <https://numpy.org>
- [7] T. Oliphant. «SciPy Documentation,» visitato il giorno 2026. indirizzo: <https://scipy.org>
- [8] dxli94. «WLASL: A large-scale dataset for Word-Level American Sign Language (WACV 20' Best Paper Honourable Mention),» visitato il giorno 2026. indirizzo: <https://github.com/dxli94/WLASL/tree/master>
- [9] SpaCy. «spaCy: "en_core_web_md" documentation.» Accessed: 2026, visitato il giorno 2026. indirizzo: https://spacy.io/models/en#en_core_web_sm
- [10] Python. «Python Download,» visitato il giorno 2026. indirizzo: <https://www.python.org/downloads/>